

Experiment No : 01

Aim : Learn basics of Numpy library for storing and efficiently processing any external data into python execution pipeline.

Theory : NumPy (short for Numerical Python) provides an efficient interface to store and operate on dense data buffers. In some ways, NumPy arrays are like Python's built-in list type, but NumPy arrays provide much more efficient storage and data operations as the arrays grow larger in size. NumPy arrays form the core of nearly the entire ecosystem of data science tools in Python

NumPy is a Python library used for working with arrays. It also has functions for working in domain of linear algebra, fourier transform, and matrices. NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

In Python we have lists that serve the purpose of arrays, but they are slow to process. NumPy aims to provide an array object that is up to 50x faster than traditional Python lists. The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy. Arrays are very frequently used in data science, where speed and resources are very important.

NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently. This behavior is called locality of reference in computer science. This is the main reason why NumPy is faster than lists. Also it is optimized to work with latest CPU architectures.

```
import numpy
numpy.__version__
```

```
'2.0.2'
```

A Python List Is More Than Just a List

Let's consider now what happens when we use a Python data structure that holds many Python objects. The standard mutable multi-element container in Python is the list. We can create a list of integers as follows:

```
L = list(range(10))
L
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
type(L[0])
```

```
int
```

```
L2 = [str(c) for c in L]
L2
```

```
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

```
type(L2[0])
```

```
str
```

```
L3 = [True, "2", 3.0, 4]
[type(item) for item in L3]
```

```
[bool, str, float, int]
```

Fixed-Type Arrays in Python

Python offers several different options for storing data in efficient, fixed-type data buffers. The built-in array module (available since Python 3.3) can be used to create dense arrays of a uniform type:

```
import array
L = list(range(10))
A = array.array('i', L)
A
```

```
array('i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Creating Arrays from Python Lists

First, we can use `np.array` to create arrays from Python lists:

```
import numpy as np
# integer array:
np.array([1, 4, 2, 5, 3])
```

```
array([1, 4, 2, 5, 3])
```

Remember that unlike Python lists, NumPy is constrained to arrays that all contain the same type. If types do not match, NumPy will upcast if possible (here, integers are up-cast to floating point):

```
np.array([3.14, 4, 2, 3])
```

```
array([3.14, 4. , 2. , 3. ])
```

If we want to explicitly set the data type of the resulting array, we can use the `dtype` keyword:

```
np.array([1, 2, 3, 4], dtype='float32')
```

```
array([1., 2., 3., 4.], dtype=float32)
```

Finally, unlike Python lists, NumPy arrays can explicitly be multi-dimensional; here's one way of initializing a multidimensional array using a list of lists:

```
# nested lists result in multi-dimensional arrays
np.array([range(i, i + 3) for i in [2, 4, 6]])
```

```
array([[2, 3, 4],
       [4, 5, 6],
       [6, 7, 8]])
```

Creating Arrays from Scratch

Especially for larger arrays, it is more efficient to create arrays from scratch using routines built into NumPy. Here are several examples:

```
# Create a length-10 integer array filled with zeros
np.zeros(10, dtype=int)
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
# Create a 3x5 floating-point array filled with ones
np.ones((3, 5), dtype=float)
```

```
array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
```

```
# Create a 3x5 array filled with 3.14
np.full((3, 5), 3.14)
```

```
array([[3.14, 3.14, 3.14, 3.14, 3.14],
       [3.14, 3.14, 3.14, 3.14, 3.14],
       [3.14, 3.14, 3.14, 3.14, 3.14]])
```

```
# Create an array filled with a linear sequence
# Starting at 0, ending at 20, stepping by 2
# (this is similar to the built-in range() function)
np.arange(0, 20, 2)
```

```
array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
```

```
# Create an array of five values evenly spaced between 0 and 1
np.linspace(0, 1, 5)
```

```
array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

```
# Create a 3x3 array of uniformly distributed
# random values between 0 and 1
np.random.random((3, 3))
```

```
array([[0.60977391, 0.26441147, 0.4742606 ],
       [0.65580008, 0.03946463, 0.01011661],
       [0.40947588, 0.62905775, 0.22822056]])
```

```
# Create a 3x3 array of normally distributed random values
# with mean 0 and standard deviation 1
np.random.normal(0, 1, (3, 3))
```

```
array([[ 2.33615279, -2.43904325,  0.62544791],
       [ 2.88028196,  0.56481863,  0.82032044],
       [ 1.89324133,  0.61182251,  0.12534267]])
```

```
# Create a 3x3 array of random integers in the interval [0, 10)
np.random.randint(0, 10, (3, 3))
```

```
array([[0, 1, 4],
       [9, 5, 1],
       [2, 0, 7]])
```

```
# Create a 3x3 identity matrix
np.eye(5)
```

```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```

```
# Create an uninitialized array of three integers
# The values will be whatever happens to already exist at that memory location
np.empty(3)
```

```
array([0. , 0.5, 1. ])
```

NumPy Standard Data Types

NumPy arrays contain values of a single type, so it is important to have detailed knowledge of those types and their limitations. Because NumPy is built in C, the types will be familiar to users of C, Fortran, and other related languages.

The standard NumPy data types are listed in the following table. Note that when constructing an array, they can be specified using a string:

```
np.zeros(10, dtype='int16')
```

Or using the associated NumPy object:

```
np.zeros(10, dtype=np.int16)
```

Following table shows all datatypes for Numpy Array

NumPy Array Attributes

We will learn about important Attributes with NumPy Array objects

Each array object has attributes `ndim` (the number of dimensions), `shape` (the size of each dimension), and `size` (the total size of the array), the `dtype`, the data type of the array :

Data type	Description
<code>bool</code>	Boolean (True or False) stored as a byte
<code>int_</code>	Default integer type (same as C <code>long</code> ; normally either <code>int64</code> or <code>int32</code>)
<code>intc</code>	Identical to C <code>int</code> (normally <code>int32</code> or <code>int64</code>)
<code>intp</code>	Integer used for indexing (same as C <code>ssize_t</code> ; normally either <code>int32</code> or <code>int64</code>)
<code>int8</code>	Byte (-128 to 127)
<code>int16</code>	Integer (-32768 to 32767)
<code>int32</code>	Integer (-2147483648 to 2147483647)
<code>int64</code>	Integer (-9223372036854775808 to 9223372036854775807)
<code>uint8</code>	Unsigned integer (0 to 255)
<code>uint16</code>	Unsigned integer (0 to 65535)
<code>uint32</code>	Unsigned integer (0 to 4294967295)
<code>uint64</code>	Unsigned integer (0 to 18446744073709551615)
<code>float_</code>	Shorthand for <code>float64</code> .
<code>float16</code>	Half precision float: sign bit, 5 bits exponent, 10 bits mantissa
<code>float32</code>	Single precision float: sign bit, 8 bits exponent, 23 bits mantissa
<code>float64</code>	Double precision float: sign bit, 11 bits exponent, 52 bits mantissa
<code>complex_</code>	Shorthand for <code>complex128</code> .
<code>complex64</code>	Complex number, represented by two 32-bit floats
<code>complex128</code>	Complex number, represented by two 64-bit floats

```
#Consider following sample arrays
np.random.seed(0) # seed for reproducibility

x1 = np.random.randint(10, size=6) # One-dimensional array
x2 = np.random.randint(10, size=(3, 4)) # Two-dimensional array
x3 = np.random.randint(10, size=(3, 4, 5)) # Three-dimensional array

print("x3 ndim: ", x3.ndim)
print("x3 shape:", x3.shape)
print("x3 size: ", x3.size)
print("dtype:", x3.dtype)
```

```
x3 ndim: 3
x3 shape: (3, 4, 5)
x3 size: 60
dtype: int64
```

Other attributes include `itemsize`, which lists the size (in bytes) of each array element, and `nbytes`, which lists the total size (in bytes) of the array:

```
print("itemsize:", x3.itemsize, "bytes")
print("nbytes:", x3.nbytes, "bytes")
```

```
itemsize: 8 bytes
nbytes: 480 bytes
```

Array Indexing: Accessing Single Elements

Next we learn how to access single element in a NumPy array NumPy follows indexing similar to that of Python in a dimension index starts at 0 till length-1

So `x1[0]` will mean 0th element and `x1[5]` means sixth element in array `x1`.

We can use negative index value to indicate accessing elements from back side of array.

In a multi-dimensional array, items can be accessed using a comma-separated tuple of indices as shown in below code cell.

```
# accessing third list's first element
x2[2, 0]

np.int64(1)
```

```
# modifying value at a particular index
x2[0, 0] = 12
x2
```

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```

Array Slicing: Accessing Subarrays

Just as we can use square brackets to access individual array elements, we can also use them to access subarrays with the slice notation, marked by the colon (:) character. The NumPy slicing syntax follows that of the standard Python list; to access a slice of an array *x*, use this:

`x[start:stop:step]`

If any of these are unspecified, they default to the values `start=0`, `stop=size of dimension`, `step=1`. We'll take a look at accessing subarrays in one dimension and in multiple dimensions.

```
x = np.arange(10)
x
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
x[:5] # first five elements
```

```
array([0, 1, 2, 3, 4])
```

```
x[:5] # elements after index five
```

```
array([0, 1, 2, 3, 4])
```

```
x[4:7] # sub-array of index 4, 5, 6
```

```
array([4, 5, 6])
```

```
x[::2] # every other element
```

```
array([0, 2, 4, 6, 8])
```

```
x[2::2] # every other element starting at index 2
```

```
array([2, 4, 6, 8])
```

A potentially confusing case is when the step value is negative. In this case, the defaults for start and stop are swapped. This becomes a convenient way to reverse an array:

```
x[::-1] # all elements, reversed
```

```
array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

```
x[3::-2] # reversed every other from index 3
```

```
array([3, 1])
```

Multi-dimensional subarrays

Multi-dimensional slices work in the same way, with multiple slices separated by commas. For example:

```
x2[:2, :3] # This is sub Array of x2 with first two rows and first three columns
```

```
array([[12,  5,  2],
       [ 7,  6,  8]])
```

```
#Check how we can reverse the multidimension array
x2[::-1, ::-1]
```

```
array([[ 7,  7,  6,  1],
       [ 8,  8,  6,  7],
       [ 4,  2,  5, 12]])
```

Accessing array rows and columns

One commonly needed routine is accessing of single rows or columns of an array. This can be done by combining indexing and slicing, using an empty slice marked by a single colon (:):

```
print(x2[:, 0]) # first column of x2
```

```
[12  7  1]
```

```
print(x2[0, :]) # first row of x2
```

```
[12  5  2  4]
```

Subarrays as no-copy views

One important—and extremely useful—thing to know about array slices is that they return views rather than copies of the array data.

This is one area in which NumPy array slicing differs from Python list slicing: in lists, slices will be copies. Consider our two-dimensional array from before:

```
#Extract 2*2 sub array from x2
x2_sub = x2[:2, :2]
print(x2_sub)
```

```
[[12  5]
 [ 7  6]]
```

```
x2_sub[0, 0] = 99
#Above statement not oly modifies subarray but also the original array as well
print(x2_sub)
```

```
[[99  5]
 [ 7  6]]
```

```
import numpy as np

x2 = np.array([[1, 2],
               [3, 4]])

x2_sub_copy = x2.copy() # define it first

x2_sub_copy[0, 0] = 42
print(x2_sub_copy)
print(x2)
```

```
[[42  2]
 [ 3  4]]
[[1  2]
 [3  4]]
```

Array Concatenation and Splitting

All of the preceding routines worked on single arrays. It's also possible to combine multiple arrays into one, and to conversely split a single array into multiple arrays. We'll take a look at those operations here. Concatenation of arrays

Concatenation, or joining of two arrays in NumPy, is primarily accomplished using the routines `np.concatenate`, `np.vstack`, and `np.hstack`. `np.concatenate` takes a tuple or list of arrays as its first argument, as we can see here:

```
x = np.array([1, 2, 3])
y = np.array([3, 2, 1])
np.concatenate([x, y])
```

```
array([1, 2, 3, 3, 2, 1])
```

```
grid = np.array([[1, 2, 3],
                 [4, 5, 6]])
```

```
# concatenate along the first axis
np.concatenate([grid, grid])
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [1, 2, 3],
       [4, 5, 6]])
```

```
# concatenate along the second axis (zero-indexed)
np.concatenate([grid, grid], axis=1)
```

```
array([[1, 2, 3, 1, 2, 3],
       [4, 5, 6, 4, 5, 6]])
```

When joining arrays of mixed dimensions, it can be clearer to use the `np.vstack` (vertical stack) and `np.hstack` (horizontal stack) functions as shown below:

```
x = np.array([1, 2, 3])
grid = np.array([[9, 8, 7],
                 [6, 5, 4]])
```

```
# vertically stack the arrays
np.vstack([x, grid])
```

```
array([[1, 2, 3],
       [9, 8, 7],
       [6, 5, 4]])
```

```
# horizontally stack the arrays
y = np.array([[99],
              [99]])
np.hstack([grid, y])
```

```
array([[ 9,  8,  7, 99],
       [ 6,  5,  4, 99]])
```

Splitting of arrays

The opposite of concatenation is splitting, which is implemented by the functions `np.split`, `np.hsplit`, and `np.vsplit`. For each of these, we can pass a list of indices giving the split points:

```
x = [1, 2, 3, 99, 99, 3, 2, 1]
x1, x2, x3 = np.split(x, [3, 5])
print(x1, x2, x3)
```

```
[1 2 3] [99 99] [3 2 1]
```

Notice that `N` split-points, leads to `N + 1` subarrays. The related functions `np.hsplit` and `np.vsplit` are similar:

```
grid = np.arange(16).reshape((4, 4))
grid
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])
```

```
upper, lower = np.vsplit(grid, [2])
print(upper)
print(lower)
```

```
[[0 1 2 3]
 [4 5 6 7]]
[[ 8  9 10 11]
 [12 13 14 15]]
```

```
left, right = np.hsplit(grid, [2])
print(left)
print(right)
```

```
[[ 0  1]
 [ 4  5]
 [ 8  9]
 [12 13]]
[[ 2  3]
 [ 6  7]
 [10 11]
 [14 15]]
```

Introducing UFuncs

For many types of operations, NumPy provides a convenient interface into just this kind of statically typed, compiled routine. This is known as a vectorized operation. This can be accomplished by simply performing an operation on the array, which will then be applied to each element. This vectorized approach is designed to push the loop into the compiled layer that underlies NumPy, leading to much faster execution.

```
#Consider following loop based implementation to ind reciprocals for each element of an array
np.random.seed(0)

def compute_reciprocals(values):
    output = np.empty(len(values))
    for i in range(len(values)):
        output[i] = 1.0 / values[i]
    return output

big_array = np.random.randint(1, 100, size=1000000)
%timeit compute_reciprocals(big_array)
```

2.23 s ± 616 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

```
#Same operatoin using UFuncs applying '/' over array elements
%timeit (1.0 / big_array)
```

1.71 ms ± 240 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)

Vectorized operations in NumPy are implemented via ufuncs, whose main purpose is to quickly execute repeated operations on values in NumPy arrays. UFuncs are extremely flexible – before we saw an operation between a scalar and an array, but we can also operate between two arrays as well as multidimensional arrays.

Computations using vectorization through ufuncs are nearly always more efficient than their counterpart implemented using Python loops, especially as the arrays grow in size. Any time you see such a loop in a Python script, you should consider whether it can be replaced with a vectorized expression. Exploring NumPy's UFuncs

Ufuncs exist in two flavors: unary ufuncs, which operate on a single input, and binary ufuncs, which operate on two inputs. Array arithmetic

NumPy's ufuncs feel very natural to use because they make use of Python's native arithmetic operators. The standard addition, subtraction, multiplication, and division can all be used:

```
x = np.arange(4)
print("x      =", x)
print("x + 5 =", x + 5)
print("x - 5 =", x - 5)
print("x * 2 =", x * 2)
print("x / 2 =", x / 2)
print("x // 2 =", x // 2) # floor division
#some advanced arithmetic expression  -(x/2+1)^2
-(0.5*x + 1) ** 2
```

```
x      = [0 1 2 3]
x + 5 = [5 6 7 8]
x - 5 = [-5 -4 -3 -2]
x * 2 = [0 2 4 6]
x / 2 = [0.  0.5 1.  1.5]
x // 2 = [0 0 1 1]
array([-1.  , -2.25, -4.  , -6.25])
```

Specialized ufuncs

NumPy has many more ufuncs available, including hyperbolic trig functions, bitwise arithmetic, comparison operators, conversions from radians to degrees, rounding and remainders, and much more. A look through the NumPy documentation reveals a lot of interesting functionality.

Another excellent source for more specialized and obscure ufuncs is the submodule `scipy.special`. If you want to compute some obscure mathematical function on your data, chances are it is implemented in `scipy.special`. There are far too many functions to list them all, but the following snippet shows a couple that might come up in a statistics context:

```
#importing package special from scipy package
from scipy import special
```

```
# Gamma functions (generalized factorials) and related functions
x = [1, 5, 10]
print("gamma(x)      =", special.gamma(x))
print("ln|gamma(x)| =", special.gammaln(x))
print("beta(x, 2)   =", special.beta(x, 2))
```

```
gamma(x)      = [1.0000e+00 2.4000e+01 3.6288e+05]
ln|gamma(x)| = [ 0.          3.17805383 12.80182748]
beta(x, 2)   = [0.5          0.03333333 0.00909091]
```

```
# Many other special functions like error functions, beta integral can also be evaluated.
# Aggregates
# For binary ufuncs, there are some interesting aggregates that can be computed directly
# from the object. For example, if we'd like to reduce an array with a particular operation,
# we can use the reduce method of any ufunc.
# A reduce repeatedly applies a given operation to the elements of an array until only a
# single result remains.
```

```
x = np.arange(1, 6)
np.add.reduce(x)
```

```
np.int64(15)
```

```
np.multiply.reduce(x)
```

```
np.int64(120)
```

```
#note the difference in output with accumulate
np.add.accumulate(x)
```

```
array([ 1,  3,  6, 10, 15])
```

Answer Following Questions.

Q1.What are UFuncs in numpy?

Answer:-UFuncs, or universal functions, in NumPy are vectorized operations that quickly execute repeated operations on values in NumPy arrays. They are implemented in a compiled layer underlying NumPy, leading to much faster execution compared to Python loops, especially for larger arrays. UFuncs come in two main flavors: unary ufuncs, which operate on a single input, and binary ufuncs, which operate on two inputs. They make use of Python's native arithmetic operators (like addition, subtraction, multiplication, and division), and there are many more specialized ufuncs available for various mathematical functions.

Q2.Which are various attributes of numpy arrays object?

Answer:-NumPy array objects have several important attributes that describe their characteristics:

ndim: The number of dimensions (axes) of the array. shape: A tuple indicating the size of each dimension. size: The total number of elements in the array. dtype: The data type of the elements in the array (e.g., int64, float32). itemsize: The size in bytes of each element in the array. nbytes: The total size in bytes consumed by the array data.

Q3.If you have 3 dimensional array in numpy object Obj how to identify its size, type and dimentions ? answer

Answer:- can identify the size, type, and dimensions of a 3-dimensional NumPy array using its attributes.

The code has executed successfully, creating a sample 3-dimensional NumPy array named Obj.

The output shows the following attributes of the array:

Number of dimensions (Obj.ndim): 3, confirming it's a 3-dimensional array. Shape (Obj.shape): (3, 4, 5), meaning the array has 3 'layers', each layer has 4 'rows', and each row has 5 'columns'. Total number of elements (Obj.size): 60, which is the product of its dimensions ($3 * 4 * 5 = 60$). Data type of elements (Obj.dtype): int64, indicating that all elements in the array are 64-bit integers. This demonstrates how you can easily access these properties using the .ndim, .shape, .size, and .dtype attributes of a NumPy array object.

```
import numpy as np

# Create a sample 3-dimensional array (Obj)
np.random.seed(42) # for reproducibility
Obj = np.random.randint(0, 100, size=(3, 4, 5))

print("Array Object (Obj):\n", Obj)
print("\nNumber of dimensions (Obj.ndim):", Obj.ndim)
print("Shape (Obj.shape):", Obj.shape)
print("Total number of elements (Obj.size):", Obj.size)
print("Data type of elements (Obj.dtype):", Obj.dtype)
```

```
Array Object (Obj):
[[[51 92 14 71 60]
  [20 82 86 74 74]
  [87 99 23  2 21]
  [52  1 87 29 37]]
```

```
[[[ 1 63 59 20 32]
  [75 57 21 88 48]
  [90 58 41 91 59]
  [79 14 61 61 46]]
```

```
[[[61 50 54 63  2]
  [50  6 20 72 38]
  [17  3 88 59 13]
  [ 8 89 52  1 83]]]
```

```
Number of dimensions (Obj.ndim): 3
Shape (Obj.shape): (3, 4, 5)
Total number of elements (Obj.size): 60
Data type of elements (Obj.dtype): int64
```

Q4.Consider array object named Obj with dimentions 342 and you want to convert it to 46 shape explain how will you do it ? Show this using a code cell, taking an example of 34*2 np array of random int between 1 and 50.

Answer:-To convert an array object named Obj with dimensions 3x4x2 to a 4x6 shape, you can use the reshape() method in NumPy. The key is that the total number of elements in the array must remain the same before and after the reshape operation. In this case, $3 * 4 * 2 = 24$ elements, and $4 * 6 = 24$ elements, so it's a valid reshape.

create an example array and demonstrate the reshaping in the code cell.

```
import numpy as np

# Create a 3x4x2 NumPy array with random integers between 1 and 50
np.random.seed(42) # for reproducibility
Obj = np.random.randint(1, 51, size=(3, 4, 2))

print("Original Obj array:\n", Obj)
print("Original Obj shape:", Obj.shape)
print("Total elements in Obj:", Obj.size)

# Reshape Obj to a 4x6 array
Obj_resaped = Obj.reshape(4, 6)

print("\nResaped Obj array (4x6):\n", Obj_resaped)
print("Resaped Obj shape:", Obj_resaped.shape)
print("Total elements in resaped Obj:", Obj_resaped.size)
```

```
Original Obj array:
[[[39 29]
  [15 43]
  [ 8 21]]]
```

```
[39 19]]

[[23 11]
 [11 24]
 [36 40]
 [24  3]]

[[22  2]
 [24 44]
 [30 38]
 [ 2 21]]]
Original Obj shape: (3, 4, 2)
Total elements in Obj: 24

Reshaped Obj array (4x6):
[[39 29 15 43  8 21]
 [39 19 23 11 11 24]
 [36 40 24  3 22  2]
 [24 44 30 38  2 21]]
Reshaped Obj shape: (4, 6)
Total elements in reshaped Obj: 24
```

Q5.Consider above 46 array, *sample 23* sub array from left bottom of this array store the result in variable named subObjCpy.

Answer:-To get a 2x3 sub-array from the left bottom of the 4x6 reshaped array, you'll select the last two rows and the first three columns. I'll store the result in a variable named subObjCpy.

The code executed successfully. a 2x3 sub-array from the left bottom of Obj_resaped was extracted and stored into subObjCpy.

The output shows:

The Obj_resaped array, which is the 4x6 array. The subObjCpy array, which contains the elements [[36 40 24], [24 44 30]]. This corresponds to the last two rows (indices 2 and 3) and the first three columns (indices 0, 1, and 2) of the Obj_resaped array.

```
# The 'Obj_resaped' array has shape (4, 6).
# To get a 2x3 sub-array from the left bottom, we need the last two rows (index 2 and 3) and the first three columns (index 0, 1, and 2)
subObjCpy = Obj_resaped[2:, :3]

print("Original Obj_resaped array:\n", Obj_resaped)
print("\n2x3 sub-array from left bottom (subObjCpy):\n", subObjCpy)

Original Obj_resaped array:
[[39 29 15 43  8 21]
 [39 19 23 11 11 24]
 [36 40 24  3 22  2]
 [24 44 30 38  2 21]]

2x3 sub-array from left bottom (subObjCpy):
[[36 40 24]
 [24 44 30]]
```

Q6.Make changes in 1 perticular value in the subObjCpy. Check values of Obj subObjCpy to check impact of this change of element in original and copied object. What do you conclude?

Answer:-change to a particular value in subObjCpy and then check the values of Obj_resaped and subObjCpy to see the impact. This will help us understand if subObjCpy is a view or a copy of a portion of Obj_resaped.

The code executed successfully, and the results clearly demonstrate an important characteristic of NumPy array slicing.

After modifying a value in subObjCpy (e.g., setting subObjCpy[0, 0] = 999):

The subObjCpy itself shows the updated value. Crucially, Obj_resaped (the 4x6 array from which subObjCpy was taken) also reflects this change at the corresponding position. Furthermore, the original 3-dimensional Obj array also shows the modification in the elements that correspond to the changed sub-array portion.

```
# Change a value in subObjCpy
# For example, let's change the element at index [0, 0] (first row, first column) of subObjCpy
subObjCpy[0, 0] = 999

print("Modified subObjCpy:\n", subObjCpy)
print("\nObj_resaped after modification of subObjCpy:\n", Obj_resaped)
print("\nObj after modification of subObjCpy (full array - might be large):\n", Obj)

Modified subObjCpy:
[[999 40 24]
 [ 24 44 30]]

Obj_resaped after modification of subObjCpy:
[[ 39 29 15 43  8 21]
 [ 39 19 23 11 11 24]
 [999 40 24  3 22  2]
 [ 24 44 30 38  2 21]]
```

```
Obj after modification of subObjCpy (full array - might be large):
[[[ 39  29]
   [ 15  43]
   [  8  21]
   [ 39  19]]

  [[ 23  11]
   [ 11  24]
   [999  40]
   [ 24   3]]

  [[ 22   2]
   [ 24  44]
   [ 30  38]
   [  2  21]]]
```

Q7.If you were to filter sub array view from above Obj object and want to apply modification in original and/or sub obj change respective element in other how will you achieve it?

Answer:-= Since slices already create views, any modification to a filtered view obtained from such a slice will also modify the original array's corresponding elements. If you only want to modify the sub-array and not the original, you'd explicitly create a copy using `.copy()`.

Let's demonstrate how modifying an element in a filtered view affects both the view and the original arrays.

The code executed successfully, and the results further illustrate the view behavior in NumPy.

After creating `region_for_filter` from `Obj_resaped` (which itself was a reshape of `Obj`) and then modifying an element within `region_for_filter` (e.g., changing `region_for_filter[0, 1]` from 40 to 555):

The `region_for_filter` array clearly shows the updated value. The `Obj_resaped` array, from which `region_for_filter` was derived, also reflects this change at the corresponding position (`Obj_resaped[2, 1]` is now 555). The original `Obj` array also shows this modification in its corresponding elements.

```
import numpy as np

# Ensure Obj and Obj_resaped are available from previous steps
# For demonstration, recreating with the last known state after previous modifications
Obj = np.array([[[ 39,  29],
                  [ 15,  43],
                  [  8,  21],
                  [ 39,  19]],

                [[ 23,  11],
                  [ 11,  24],
                  [999,  40],
                  [ 24,   3]],

                [[ 22,   2],
                  [ 24,  44],
                  [ 30,  38],
                  [  2,  21]]])

Obj_resaped = np.array([ [ 39,  29, 15,  43,  8,  21],
                          [ 39,  19, 23,  11, 11,  24],
                          [999,  40, 24,   3, 22,   2],
                          [ 24,  44, 30,  38,  2,  21]])

print("Initial Obj_resaped:\n", Obj_resaped)

# 1. Create a filtered sub-array view (using boolean indexing)
# Let's say we want to target elements in the last two rows and first three columns (similar to subObjCpy's area)
# and filter for elements greater than 30 within that region.

# Define the region of interest for filtering (last two rows, first three columns)
region_for_filter = Obj_resaped[2:, :3]

# Create a boolean mask for filtering within this region
# For example, let's find elements in 'region_for_filter' that are greater than 30
mask = region_for_filter > 30

print("\nRegion for filter (last two rows, first three columns of Obj_resaped):\n", region_for_filter)
print("\nBoolean mask (elements > 30 in the region):\n", mask)

# Apply the mask to get a filtered view. When using boolean indexing, the result is a copy if the indices are not consecutive.
# However, if we assign *back* to the boolean-indexed view, it modifies the original.
# Let's directly assign a new value to an element identified by the mask or a specific index within the region.

# Let's take the specific element at index [0,1] of 'region_for_filter' which is 40 and change it
# This element is Obj_resaped[2, 1]
```

```
region_for_filter[0, 1] = 555 # Change the value 40 to 555

print("\nModified region_for_filter:\n", region_for_filter)
print("\nObj_resaped after modifying region_for_filter:\n", Obj_resaped)
print("\nObj after modifying region_for_filter (full array):\n", Obj)

# Conclusion will be provided in a separate markdown cell after execution
```

```
Initial Obj_resaped:
[[ 39  29  15  43   8  21]
 [ 39  19  23  11  11  24]
 [999  40  24   3  22   2]
 [ 24  44  30  38   2  21]]

Region for filter (last two rows, first three columns of Obj_resaped):
[[999  40  24]
 [ 24  44  30]]

Boolean mask (elements > 30 in the region):
[[ True  True False]
 [False  True False]]

Modified region_for_filter:
[[999 555  24]
 [ 24  44  30]]

Obj_resaped after modifying region_for_filter:
[[ 39  29  15  43   8  21]
 [ 39  19  23  11  11  24]
 [999 555  24   3  22   2]
 [ 24  44  30  38   2  21]]

Obj after modifying region_for_filter (full array):
[[[ 39  29]
 [ 15  43]
 [   8  21]
 [ 39  19]]

 [[ 23  11]
 [ 11  24]
 [999  40]
 [ 24   3]]

 [[ 22   2]
```